
cidrtools-cffi API Reference

Release 1.0.0

Gene C

Aug 20, 2026

PYTHON API REFERENCE:

1	Features	3
2	Performance Benchmarks	5
3	Installation	7
4	Usage Example	9
4.1	High-Level API	9
5	API Reference	11
6	AI Tooling	13
7	API Reference	15
7.1	class cidrtools	15
8	Appendix	19
8.1	Installation	19
8.2	Dependencies	19
8.3	Philosophy	19
	Python Module Index	21
	Index	23

===== Cidrtools CFFI Documentation =====

Python CFFI bindings for the high-performance [cidrtools](#) C library.

This module provides a suite of tools to examine and manipulate network cidr blocks. It has the same functionality as the *cidrtools* library.

The source is available in [Github cidrtools-cffi](#) as well as [Arch AUR](#)

This package exposes the complete *cidrtools* public API using a modular, split-file architecture. It provides an optimized ABI-mode bridge to the underlying C structures alongside an ergonomic, object-oriented Python layer.

FEATURES

- **Zero-Overhead Performance:** Runs complex operations (like block compacting) at 100% native C speeds.
- **Highly Maintainable:** Written entirely in pure Python via CFFI ABI mode—no complex Cython generation or compilation needed.
- **Modern Tooling Pipeline:** Fully managed by uv, meson, and meson-python.
- **Robust Object Interface:** Complete memory handling, automatic garbage collection hooks, and native operator overloading.

PERFORMANCE BENCHMARKS

The following benchmarks measure the execution time required to process a random sample of **100,000 subnets** and compact them down to **54,941 subnets**.

Compared to native Python architectures, this CFFI implementation achieves a near **80x performance increase**:

Version Used	Seconds	Relative
-----	-----	-----
(a) Raw C-Code	0.011892861	1.0 x
(b) CFFI Wrapper	0.011995342	1.0 x
(c) Cython/C Bridge	0.021802806	1.8 x
(d) Pure Cython	0.026872699	2.3 x
(e) Pure Python	0.935538276	78.7 x

*100,000 subnets compacted to 54,941 subnets

INSTALLATION

To develop or use this library locally within your virtual environment managed by `uv`, execute the following commands in the project root:

```
# Sync environment dependencies
uv sync

# Install the package in editable development mode
uv pip install -e .

# Execute the pytest suite to verify your setup
uv run pytest
```


USAGE EXAMPLE

4.1 High-Level API

```
from cidrtools import CidrBlock, CidrBlocks

# Instantiating one cidr block from a string.
cidr = CidrBlock("192.168.1.0/24")
print(f"IPs within network: {cidr.num_ips}")

# Check if ip or cidr is in the cidr subnet.
ip_str = "192.168.1.50"
is_in = "is" if (ip_str in cidr) else "is not"
print(f' {ip_str} {is_in} in {cidr}')
```

=> 192.168.1.50 is in 192.168.1.0/24

```
cidrs = CidrBlocks(["192.168.1.0/23", "10.0.0.0/24"])
is_subnet = "is" if cidr.is_subnet_of(cidrs) else "is not"
print(f' {str(cidr)} {is_subnet} a subnet of {list(cidrs)}')
```

=> 192.168.1.0/24 is a subnet of ['192.168.0.0/23', '10.0.0.0/24']

```
cidrs.add('10.0.2.0/24')
list(cidrs)
```

=> ['192.168.0.0/23', '10.0.0.0/24', '10.0.2.0/24']

```
cidrs.compact()
list(cidrs)
```

=> ['10.0.0.0/24', '10.0.2.0/24', '192.168.0.0/23']

```
networks = CidrCollection(["192.168.1.0/25", "192.168.1.128/25"])
networks.sort()
networks.compact()
```

(continues on next page)

(continued from previous page)

```
# back to a standard Python list of strings
print(list(networks)) # Output: ['192.168.1.0/24']
```

API REFERENCE

Documentation is built using Sphinx and the PDF manual is found in *src/data/docs* and installed to */usr/share/cidrtools-cffi*.

To create a local copy of the html and PDF documentation:

```
cd src/data/docs  
./build-manual.sh
```


AI TOOLING

Assisted by:

- Anthropic's Claude
- Google's Gemini

API REFERENCE

This section contains the API documentation for the `cidrtools` package.

7.1 class `cidrtools`

cidrtools-cffi package provides:

`cidrtools` module: - public class/member functions - wrapper function for each `cidrtool` `ct_xx` function.

`class cidrtools.CidrBlock(cidr_str: str)`

Bases: `object`

Class to manage one `cidr` block. Uses `cidrtools` C-library `CtCidrBlock` structure.

Handles memory allocation, type inspections, range mapping, and contains (is-subnet).

Built on the `cidrtools` library.

`cidr_parts()` → `tuple[str, int]`

Returns the IP address and prefix as a tuple.

- (`ip_address: str`, `prefix: int`)

`clean()` → `bool`

Clean up invalid `cidr` blocks (`prefix`, `host bits` being set)

Returns

True on success otherwise False

`fix_host_bits()` → `int`

Cleans up `cidr` with `host bits` set, by setting all `host bits` to zero.

`format_host_bits()` → `str`

Returns a formatted “address” string of the `host bits` Identical to `get_host_bits()` -

`get_host_bits()` → `str`

Extracts the underlying address portion - these are the bits past the `network bits` defined by the `prefix length`.

<`prefix length` of `network-bits` ... ><`host-bits`>

E.g. `10.2.2.3/24 => 0.0.0.3` Note that the `host bits` are tracked internally but the `cidr` is always “fixed” Example: - `cidr = CidrBlock(“192.168.1.22/24”) - str(cidr) -> “192.168.1.0/24” - cidr.get_host_bits() -> “0.0.0.22”`

`increment_by(steps: int) → str`

Find a new network address advanced by 'steps' IP addresses.

property `is_ipv4: bool`

Returns True if the cidr block is an IPv4 network family

Returns

True if the network family is IPv4, False otherwise.

property `is_ipv6: bool`

Returns True if the cidr block is an IPv6 network family.

Returns

True if the network family is IPv6, False otherwise.

`is_subnet_of(cidrs: CidrBlocks) → bool`

Checks if this block is a subnet of any of the collection of cidrs.

property `num_ips: int`

Returns the total number of IPs inside this cidr block.

Note that for IPv6 this is capped at 64 bit interger max.

`set_prefix(prefix: int) → int`

Modifies the network prefix to be the new value.

`split(prefix: int) → CidrBlocks`

Splits the cidr block into a collection of smaller subnets using the provided prefix.

`to_range() → tuple[str, str]`

Computes the first and last IP addresses of this cidr block. See also `to_range_mid()`

`to_range_mid() → tuple[str, str, str]`

Compute the first, middle and last IP addresses of this cidr block.

`to_string() → str`

Return the cidr string. Same as `str(self)`

`class cidrttools.CidrBlocks(initial_cidrs: list[str] | None = None)`

Bases: `object`

An ordered list of `CtCidrs`

A list of cidr blocks. Supports a variety of operations via the member functions such as: - compacting - filtering (excluding), - slicing, - cleaning.

Built on the cidrttools library. Manages memory allocations.

`add(cidr: CidrBlock | str) → bool`

Appends a CIDR block to the collection.

Parameters

`cidr` – The cidr block (object or raw string) to add.

Returns

True if the append succeeded else false.

`clean() → bool`

Clean any invalid cidr blocks (prefix, host bits being set)

Returns

True if success else False

`compact()` → int

Compress the list of cidrs to the minimal number of subnets.

This combines adjacent or overlapping subnets to make the smallest number of subnets. This is done in place.

Returns

Return status code from the underlying C function.

`exclude(excluded_cidrs: CidrBlocks)` → int

Exclude a set of cidr blocks. This is done in place.

new cidrs => cidrs - excluded_cidrs.

Parameters

`excluded_cidrs` – The cidrs to be excluded.

Returns

Return status code of the underlying C library.

`sort()` → int

In-place sort of the cidr blocks.

Returns

Return status code from the underlying C library

`split_by_family()` → tuple[CidrBlocks, CidrBlocks]

Splits the cidr blocks into separate IPv4 and IPv6 lists.

Returns

tuple of cidr lists - (IPv4_Pool, IPv6_Pool).

Raises

`RuntimeError` – If errors found by C-library

`to_strings()` → list[str]

Returns a list of cidr strings. Optimized for speed.

`class cidrtools.CidrTools`

Bases: object

Class with some additional member functions of the *cidrtools* library.

Underlying cidrtools version and some DNS convenience functions.

`static ct_version()` → str

Returns the native C library version target compilation string.

Returns

The loaded library system release semantic tag value.

Return type

str

`static hostname_to_addresses(hostname: str)` → CidrBlocks

Resolves a hostname string into a structured collection of CIDR blocks.

Parameters

`hostname (str)` – The explicit domain URL path target string to resolve.

Returns

A fresh collection array tracking resolved system addresses blocks.

Return type

CidrBlocks

Raises

`RuntimeError` – If standard resolution paths fail or return unexpected status flags.

`static ip_str_to_hostname(ip_str: str) → str`

Performs a reverse DNS lookup mapping an active IP address string back to a hostname.

Parameters

`ip_str (str)` – The textual raw representation layout of the target lookup address.

Returns

The fully qualified resolved host identifier name string.

Return type

`str`

Raises

`RuntimeError` – If reverse lookup actions time out or tracking channels fail.

`static range_to_cidrs(first_ip: str, last_ip: str) → CidrBlocks`

Converts an IP range to a minimal collection of CIDR blocks.

Should this be moved to `CidrBlocks.range_to_cidrs()` ?

Parameters

- `first_ip` – The first network IP of the range
- `last_ip` – The last IP of the range

Returns

A minimal representation tracking collection of matched network blocks.

Raises

- `ValueError` – If input strings map incorrectly to standard protocol types.
- `RuntimeError` – If underlying C matrix calculations encounter configuration faults.

8.1 Installation

On Arch you can build using the provided PKGBUILD in the packaging directory or from the AUR. To build manually, clone the repo and :

```
rm -f dist/*
./scripts/do-build
./scripts/do-install [destination]
```

The default destination is *build/pkg*.

8.2 Dependencies

Run Time :

- cidrtools
- python-cffi
- python (3.14 or later)

Building Package :

- git
- meson
- meson-python
- uv
- python-uv-build
- rsync
- python-pytest
- python-pytest-asyncio

8.3 Philosophy

We follow the *live at head commit* philosophy as recommended by Google's Abseil team¹. This means we recommend using the latest commit on git master branch.

¹ <https://abseil.io/about/philosophy#upgrade-support>

PYTHON MODULE INDEX

C

`cidrtools`, [15](#)

A

`add()` (*cidrtools.CidrBlocks method*), 16

C

`cidr_parts()` (*cidrtools.CidrBlock method*), 15

`CidrBlock` (class in *cidrtools*), 15

`CidrBlocks` (class in *cidrtools*), 16

`cidrtools`
module, 15

`CidrTools` (class in *cidrtools*), 17

`clean()` (*cidrtools.CidrBlock method*), 15

`clean()` (*cidrtools.CidrBlocks method*), 16

`compact()` (*cidrtools.CidrBlocks method*), 17

`ct_version()` (*cidrtools.CidrTools static method*), 17

E

`exclude()` (*cidrtools.CidrBlocks method*), 17

F

`fix_host_bits()` (*cidrtools.CidrBlock method*), 15

`format_host_bits()` (*cidrtools.CidrBlock method*), 15

G

`get_host_bits()` (*cidrtools.CidrBlock method*), 15

H

`hostname_to_addresses()` (*cidrtools.CidrTools static method*), 17

I

`increment_by()` (*cidrtools.CidrBlock method*), 15

`ip_str_to_hostname()` (*cidrtools.CidrTools static method*), 18

`is_ipv4` (*cidrtools.CidrBlock property*), 16

`is_ipv6` (*cidrtools.CidrBlock property*), 16

`is_subnet_of()` (*cidrtools.CidrBlock method*), 16

M

module
 cidrtools, 15

N

`num_ips` (*cidrtools.CidrBlock property*), 16

R

`range_to_cidrs()` (*cidrtools.CidrTools static method*), 18

S

`set_prefix()` (*cidrtools.CidrBlock method*), 16

`sort()` (*cidrtools.CidrBlocks method*), 17

`split()` (*cidrtools.CidrBlock method*), 16

`split_by_family()` (*cidrtools.CidrBlocks method*), 17

T

`to_range()` (*cidrtools.CidrBlock method*), 16

`to_range_mid()` (*cidrtools.CidrBlock method*), 16

`to_string()` (*cidrtools.CidrBlock method*), 16

`to_strings()` (*cidrtools.CidrBlocks method*), 17